

Sezione 6/6 — Error handling, observability, testing

Cosa succede quando va male, come ce ne accorgiamo, come ci assicuriamo che funzioni

6.1 Tassonomia degli errori

Categoria	Esempi	Strategia
Network	Datacenter unreachable, DNS fail, TLS handshake fail, timeout	Coda offline + retry esponenziale (2/4/8/16/30/60s, max 1h). Notifica utente solo dopo 3 tentativi falliti consecutivi (no rumore).
Auth	API key scaduta, 401/403, OAuth token expired	Sospensione coda, dialog "Re-authenticate". Non si svuota la coda, riparte dopo nuovo login.
Concorrenza	409 Conflict, parent_revision stale	Flusso ConflictResolver (sezione 4.3). Mai swallow del conflitto.
Schema validation	422 Unprocessable, GraphML malformato, vincolo CIDOC violato	Errore non recoverable automatico. Dialog mostra le violazioni puntuali e suggerisce edit. Push bloccato fino a fix.
Integrity	Checksum SHA-256 non combacia post-download, GraphML corrotto, JSON vocab parser fail	Reject del payload, retry automatico una volta, poi log error e dialog all'utente.
DB	SQLite locked, PostgreSQL connection refused, query timeout	Reuso del pyarchinit_db_manager esistente. Errori bubble-up con context "during sync operation X".
Filesystem	<code>paradata.graphml</code> read-only, disco pieno, permission denied su <code>~/.cache/pyarchinit/</code>	Notifica utente con path specifico, suggerisce fix. Niente crash, fallback a in-memory + warning persistente.
Bug interno	Eccezione non gestita nel SyncEngine, GraphMerger throw	Wrapper top-level cattura → log con stack + payload anonimizzato → dialog "Open issue with logs". Non blocca pyarchinit funzionalità non-sync.

PRINCIPIO GUIDA

Lo strato sync è **secondario** al lavoro di scavo. Un errore di sync non deve mai bloccare il salvataggio di una scheda US, l'apertura del progetto, o la generazione della Harris matrix locale. Il SyncEngine vive in un thread separato (QThread); le sue eccezioni non risalgono al main thread di QGIS.

6.2 Logging e diagnostica

Canale	Cosa logga	Visibile a
<code>QgsMessageLog</code> tag <code>"PyArchInit-Sync"</code>	Eventi human-readable: "Push to datacenter started", "Conflict on graph X", "Vocabulary refreshed (3 new types)"	Utente via Log Messages panel di QGIS
<code>~/.cache/pyarchinit/sync.log</code> (rotating, 5 file x 2MB)	Log strutturato JSON: timestamp, op, status, latency_ms, payload_size, request_id, error_class, stack	Sviluppatore quando si chiede "share logs"
<code>~/.cache/pyarchinit/sync_queue.sqlite</code>	Coda offline: timestamp, op, payload, attempts, last_error, next_retry_at	Pulsante "Show pending sync" in status bar
Status bar QGIS	Indicatore stato live: idle / syncing / pending(N) / conflict / error. Click apre dettaglio.	Utente sempre

I log **non contengono dati personali** (non si logga il contenuto stratigrafico, solo metriche e ID). Il payload completo è solo in `sync_queue.sqlite`, esportabile dall'utente con consent esplicito per debugging.

6.3 Metriche locali (no telemetry esterna)

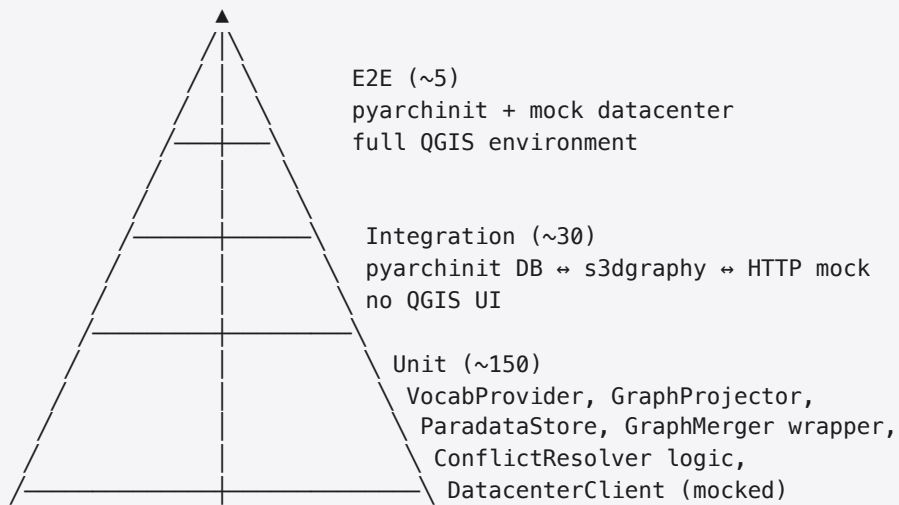
Una piccola tabella SQLite locale `~/.cache/pyarchinit/sync_metrics.sqlite` conta:

- `op_count` per tipo (push/pull/diff/vocab_refresh)
- `error_count` per tipo (network/auth/conflict/validation/...)
- `conflict_resolution_outcome` (local/remote/branched)
- `p50/p95/p99` latency per op
- `queue_depth_high_water_mark`

Esposte in un dialog "Sync diagnostics" per troubleshooting. **Niente invio automatico** a server esterni.

6.4 Strategia di test — piramide

Distribuzione test (target)



6.5 Test per modulo

Modulo	Cosa coprire	Esempi
VocabProvider	Lettura JSON, override priorità, hot-reload, missing translation fallback, signal emission	"new type added → comboBox shown after reload", "missing IT label → fallback to EN with warning logged"
GraphProjector	Idempotenza, mapping completo per ogni unita_tipo, gestione record orfani, geometrie spaziali	"project twice → identical Graph", "us_table con 1500 record → graph con 1500 nodi tipizzati"
ParadataStore	CRUD AuthorNode/EpochNode/etc, persistenza atomica (no graphml a metà su crash), import esistente	"add author + crash before save → next open shows no author", "atomic via tempfile + rename"
SyncEngine	Push happy path, push 409→pull→merge→push, offline queue, retry backoff, idempotenza request_id	"two retries with same request_id → server returns same revision", "queue persists across QGIS restart"
DatacenterClient	Backend EM mock, backend GraphDB con triple-store locale (Apache Jena Fuseki in docker), auth provider switching	"swap from EMBackend to GraphDBBackend → SyncEngine continues to work without changes"
ConflictResolver	Logic puro (UI separata), policy per tipo nodo, output deterministico	"conflict on LicenseNode → server-wins applied", "conflict on US → returns 'requires user'"
Migrazione vocab	Dry-run report accurato, apply idempotente, rollback funzionante, multi-DB	"USVA→USVs su 200 record, idempotenza: re-run 0 changes", "rollback ripristina USVA originali"
UUID backfill	Schema migration su SQLite e PostgreSQL, partial failure recovery, indici unique funzionanti	"interrupt mid-backfill → script ricomincia dai NULL"

6.6 Test E2E con mock datacenter

Un piccolo HTTP server in `tests/e2e/mock_datacenter.py` implementa il contratto della sezione 4.4 con storage in memoria. Ogni test E2E:

1. Avvia mock datacenter su porta random
2. Crea un DB pyarchinit temporaneo con dataset sintetico
3. Configura DatacenterClient verso localhost
4. Esegue scenario (publish, conflict, vocab refresh, ecc.)
5. Asserisce stato finale di DB locale + paradata.graphml + storage mock
6. Tear down

5 scenari E2E minimi prima della release:

1. **Happy path publish** — primo push, secondo push, terzo push (3 revisioni in fila)
2. **Conflict + resolve** — A pubblica R1, B pubblica R2, A prova R1→409→pull R2→merge→push R3
3. **Offline queue** — push con datacenter down, datacenter su, retry success
4. **Vocabulary update** — datacenter pubblica nuovo tipo, pyarchinit lo riceve, comboBox lo mostra
5. **Pull-only browse** — utente B scarica grafo di utente A in read-only mode, non può push

6.7 Test su QGIS reale (smoke + manual)

Tre check manuali da fare prima di taggare la release:

1. **Smoke su 3 OS** — macOS Sequoia, Windows 11, Ubuntu 24.04 LTS. Aprire un progetto pyarchinit pre-upgrade, vedere che la migrazione si offre, applicarla, verificare che la matrice si genera ancora.
2. **Stress su DB grande** — su un dataset reale (es. Volterra, ~1500 US) misurare push latency. Target: < 5s per il GraphML completo, < 30s per l'upload con datacenter reale.
3. **UI responsiveness durante sync** — durante un push, la UI di QGIS deve restare fluida (sync in QThread, non blocca event loop).

6.8 Definition of Done

- Tutti i test unit + integration passano in CI (target: GitHub Actions, 3 OS × 2 Python = 6 job)
- Test E2E (5 scenari) passano contro il mock datacenter
- Smoke test manuale su 3 OS documentato in `docs/superpowers/specs/s3dgraphy-bridge-design/smoke-results.md`
- CHANGELOG aggiornato con entry bilingue (IT/EN)
- Tutorials aggiornati nelle 9 lingue per la nuova UI di sync
- Versione plugin bumpata, tag git creato, push origin
- Documentazione API contract per il team EM Datacenter consegnata e riconosciuta